

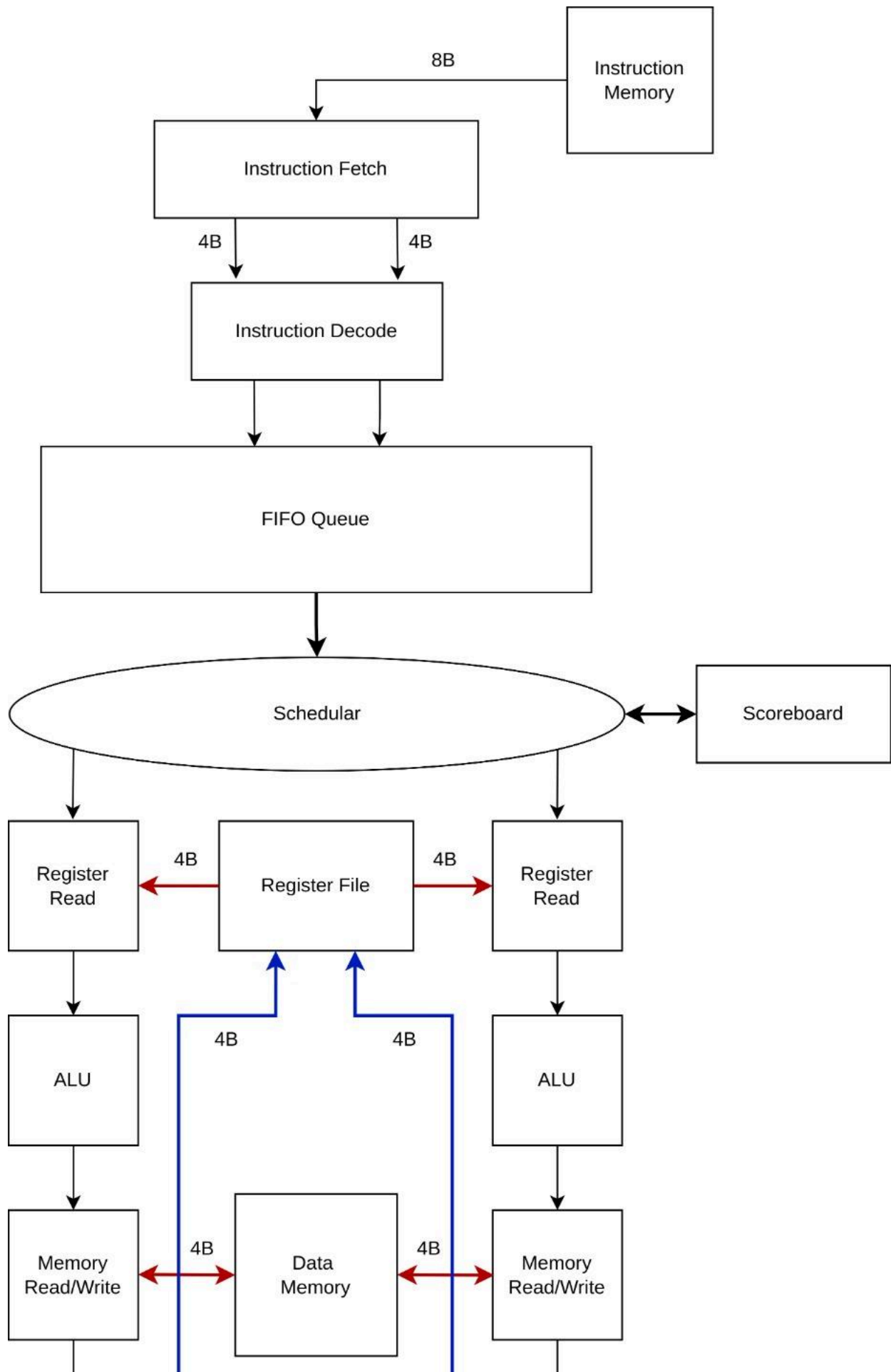


In-Order Processor Design

This doc explains the design of processor at module level. Make the modules as described below. The description given provides a fair idea of each module although there is enough room for exploration and design which can be explored by you guys.

Overall flow:

The diagram below presents the overall flow of the design to get an overall idea.



Individual Module Details.

1. Instruction Fetch Module

Purpose

- Fetch **two 32-bit RV32I instructions** per cycle from instruction memory.
- Provide these instructions to the decode stage.
- Maintain and update the **Program Counter**.

Inputs

- PC (Program Counter): Current instruction address.
- Branch Taken (from EX/MEM stage): Control signal to redirect PC.
- Branch Target Address: New PC value if a branch is taken.
- Stall Signal: From downstream stages (e.g., decode or scoreboard).
- Flush Signal: To flush instructions in case of misprediction or control hazard.

Outputs

- Instr1, Instr2: Two 32-bit instructions fetched.
- PC+4, PC+8: Next instruction addresses (for tracking and debugging).
- Fetch_Valid: Indicates whether valid instructions are output.
- PC_out: Current PC value(s) for trace/debug purposes.

Internal Components

- PC Register: Holds the current instruction address.
- Instruction Memory: Read-only memory containing instruction words.
- Instruction Buffer (Optional but helpful): Temporary buffer to hold fetched instructions if the decode stage is stalled.
- Incrementer / Branch Handler: Logic to update PC based on normal execution or branch instructions.

Operation

On every clock cycle (if not stalled):

- Fetch two instructions at PC and PC + 4.
- Send them to the decode stage.
- Increment PC by 8 (to point to the next pair of instructions).

If branch is taken:

- Override PC with Branch Target Address.
- Flush invalid instructions.

If stalled:

- Hold the current PC.
- Do not fetch new instructions.

Hazards and Control

Control Hazard (Branch Misprediction):

- Detected after the decode or execute stage.
- Flush the fetch and decode stages.
- Update PC to correct branch target.

Structural Hazard:

- If decode is full or if requested by other stages, assert stall.

Instruction Alignment:

- Ensure the PC is always aligned to 4 bytes (since RV32I instructions are 32-bit).

To be noted

- PC update logic must be sensitive to both normal execution and branch redirection.
- Instruction memory can be modeled as a ROM block (case, readmemh, or synthesized ROM).
- Include pipeline registers to buffer fetched instructions before decode.
- Use enable/valid signals to gate fetch on stall conditions.

2. Instruction Decode

Decode stage will be very similar to the decode you designed for a normal pipeline just replicated twice because both of them are independent of each other and can happen in parallel.

Purpose:

- Decode two 32-bit instructions per cycle.
- Extract control signals and operands.
- Identify instruction types (ALU, branch, memory, etc.).
- Send decoded info to the FIFO queue via the pipeline register.
- Detect control hazards, instructions like jump and branch.
- Think of a way to mitigate jump from this point (Applicable for in order only).

Inputs:

- Instr1, Instr2: Two 32-bit instructions from the Fetch stage.
- PC1, PC2: Corresponding PC values of the fetched instructions.
- Stall: From scoreboard or FIFO queue indicating decode stage should pause.
- Flush: Control signal to discard decoded instructions (e.g., due to branch misprediction).

Outputs:

- Opcode, Funct3, Funct7: Control field decoding.
- rd, rs1, rs2: Destination and source register addresses.
- Immediate: Extracted immediate value (sign-extended if required).
- InstrType: Type of instruction (R/I/S/B/U/J).
- Control Signals: ALU operation, MemRead, MemWrite, Branch, RegWrite, etc.
- Decoded_Valid: Signals whether instruction is valid and ready for FIFO or scheduler.
- To_FIFO: Bundled instruction info passed to FIFO or scheduler.

Internal Components:

1. **Instruction Field Extractors:**
 - For extracting opcode, funct3, funct7, rd, rs1, rs2, imm, etc.
2. **Immediate Generator:**
 - Generates correct signed immediate value based on instruction type.
3. **Control Unit:**
 - Generates control signals for execution units (ALU/MEM/BRANCH).
4. **Dependency Checker (optional):**
 - Check rs1/rs2 dependencies between Instr1 and Instr2.

Logic:

- Parallel decode both instructions.
- Extract fields, generate immediates, and determine control signals.
- Pass decoded bundles to FIFO Queue (or Scheduler if queue-less).
- Ensure:
 - Instructions don't conflict with each other (e.g., write-after-write hazard on the same rd).
 - Instructions are valid RV32I encodings (ignore illegal encodings).

Hazards and Checks:

- **Read-after-write (RAW)** between Instr1 and Instr2 (e.g., Instr2.rs1/rs2 = Instr1.rd):
 - Can either stall Instr2, swap issue order, or mark invalid for current cycle.
- **Illegal Instructions:**
 - Raise exception or drop with Decoded_Valid = 0.

To be noted:

- Use case statements or decoder blocks to classify opcodes.
- Keep logic combinational unless you're pipelining decode.

3. FIFO Queue

Purpose:

- Temporarily **hold decoded instruction bundles** before they are issued to functional units by the Scheduler.
- Allow smoother handoff between decode and execution stages, especially if decode produces instructions faster than the scheduler can issue them.
- Enable basic in-order buffering and flow control.

Inputs:

- Decoded_Instr1, Decoded_Instr2: Decoded instruction bundles (including control signals, rs1/rs2/rd, immediate, etc.)
- Decoded_Valid1, Decoded_Valid2: Valid bits for the two decoded instructions.
- Stall_from_Scheduler: If scheduler can't accept new instructions, FIFO must not enqueue.
- Flush: Clears the queue (e.g., due to mispredicted branch or exception).

Outputs:

- FIFO_Out1, FIFO_Out2: Instruction bundles to be issued to execution units.
- FIFO_Valid1, FIFO_Valid2: Indicate if outputs are valid and can be scheduled.
- FIFO_Empty, FIFO_Full: Status flags to control upstream decode and downstream scheduler.

Internal Structure:

- Typically a circular buffer (queue with head and tail pointers).
- Stores instruction metadata: opcode, rs1, rs2, rd, immediate, control signals, PC, etc.

Operation Logic:

- **Enqueue:** On every cycle, if not stalled and space is available, enqueue 1 or 2 decoded instructions into the FIFO.
- **Dequeue:** On every cycle, if the scheduler is ready, send 1 or 2 instructions from the head of the FIFO to the scheduler.
- Maintain **in-order sequencing** by dequeuing in fetch/decode order.
- **Backpressure:** If FIFO is full, signal stall to the decode stage.

Width and Depth:

- Width: Each entry holds one instruction bundle (including all decoded fields).

- Depth: Depends on performance tradeoffs; typically 4–8 entries per superscalar lane.
 - For 2-way, you might use a shared FIFO with tagging, or two parallel FIFOs (simpler control but higher area).

Hazards and Notes:

- Instruction Pairing: Must preserve the original pair order fetched/decode (e.g., issue instr1 before instr2).
- Branch Flush: If a branch is taken and was sitting in the FIFO, all subsequent instructions must be flushed.
- Data Hazards: FIFO doesn't resolve dependencies — that's the scheduler's job.

To be noted:

- Use dual-port RAM or register arrays to implement circular queue.
- Include control logic for full/empty conditions.
- Bundle instruction metadata into a record/struct to simplify enqueue/dequeue logic.
- Use simple head and tail pointers to manage entries.

4. Scheduler

Purpose:

- Select up to two instructions per cycle from the FIFO queue.
- Issue instructions to available execution units (e.g., ALU, Memory).
- Ensure correct program order and data dependency handling.
- Maintain in-order behavior in a 2-way issue context.

Inputs:

- FIFO_Out1, FIFO_Out2: Two oldest instruction bundles from FIFO.
- FIFO_Valid1, FIFO_Valid2: Indicate instruction validity.
- RegStatusTable (Scoreboard): Tracks availability of register values.
- Execution_Unit_Ready: Status signals from ALU, Load/Store units.
- Flush: From misprediction or exception — cancel instructions.
- Retire_Writeback_Info: Used to update register readiness (rd becomes available).

Outputs:

- Issue_Instr1, Issue_Instr2: Instructions issued to execution stage.
- Issue_Valid1, Issue_Valid2: Validity flags for issued instructions.
- FIFO_Stall: If scheduler can't issue, FIFO must hold outputs.
- RegStatusTable_Update: Marks destination registers (rd) as "in use".

Internal Logic:

- **Dependency Checker:**
 - Checks if rs1 and rs2 of each instruction are ready.
- **Issue Logic:**
 - Selects up to two instructions per cycle for issuing.
 - Enforces:
 - Structural hazard checks (e.g., FU availability)
 - Data hazard checks (RAW)
 - **In-order** constraints between FIFO instructions

Scoreboard (RegStatusTable):

- 32-bit array tracking availability of each register.

Key Rules:

1. **Instr1 can issue** if:
 - Both rs1 and rs2 are ready
 - Required functional unit is free
2. **Instr2 can issue** only if:
 - **Instr1 has issued or is independent**
 - No data hazard with Instr1 (e.g., `Instr2.rs1 == Instr1.rd`)
 - Both rs1/rs2 are ready
 - Required execution unit is free
3. If **Instr1 is stalled, Instr2 must wait** — to preserve in-order semantics.

Scoreboard Updates:

- On **issue**:
 - Destination register (**rd**) is marked **busy**.
- On **writeback**:
 - Register is marked **available** again.
- This allows checking if rs1/rs2 are **ready or waiting**.

Execution Units:

- We have **2 ALUs** branches.
- Scheduler must:
 - Route instructions to the right unit.
 - Check unit availability.

Hazards and Control:

- **RAW Hazards:**

- Block issue if a source register isn't ready.
- **Structural Hazards:**
 - ALU/MEM unit not free -> stall issue.
- **WAW/WAR (not common):**
 - These do not occur in in-order processors.

To be noted:

- Represent scoreboard as a simple 32-bit register (1 bit per register).
- Use combinational logic to:
 - Check readiness of each instruction
 - Compare source/dest regs for pairwise hazard
- Use stateful logic to update scoreboard on issue and writeback.

5. Register read and ALU execution

Purpose

- Read operands (rs1, rs2) from the register file.
- Perform ALU operations such as ADD, SUB, AND, OR, SLT, etc.
- Handle branch condition evaluation (e.g., BEQ, BNE).
- Compute memory addresses for load/store (base + offset).

Inputs

- rs1, rs2: Source register addresses from issued instruction.
- rd: Destination register address.
- Immediate: Immediate operand (if applicable).
- ALU_Op: Control signal indicating the ALU function (from decode).
- Instruction_Type: Helps distinguish between branch, ALU, load/store.
- Register File: Shared register file read port(s).
- PC: For branch/jump offset calculation.

Outputs

- ALU_Result: Computed result (e.g., arithmetic result or address).
- Branch_Decision: True/False if branch is taken.
- rd: Destination register (passed forward for writeback).
- Instruction_Info: Full bundle passed to memory or writeback stage.

Internal Components

Register File:

- 2 read ports (for rs1 and rs2)
- 1 write port (handled in writeback stage)

ALU Unit:

- Executes operation defined by ALU_Op

Branch Logic Unit:

- Evaluates branch conditions ($rs1 == rs2$, $rs1 < rs2$, etc.)

Logic

- Read operand values: $rs1_val$ and $rs2_val$.
- Depending on instruction type:
 - **R-type**: $ALU(rs1_val, rs2_val)$
 - **I-type**: $ALU(rs1_val, imm)$
 - **S-type**: Compute address: $rs1_val + imm$ (for store)
 - **Branch**: Evaluate $rs1_val$ and $rs2_val$

Output result to MEM stage or directly to writeback.

Hazard and Control

Forwarding (optional):

- Not always required in in-order, but useful if writeback isn't immediate.
- Can increase complexity though.

Stalls:

- If result needed by next instruction and writeback is pending.

Take care:

- ALU is purely combinational logic — use case over ALU_Op.
- Register file can be inferred from arrays of registers ($reg[31:0]$ regs $[0:31]$ in Verilog).
- Branch logic can be separated or integrated in ALU.
- Include logic to track whether result is destined for register write or memory access.

6. Memory

Purpose:

- Handle load (e.g., LW) and store (e.g., SW) instructions.
- Interact with data memory using calculated addresses.
- Forward memory-read results to the writeback stage.

- Maintain in-order execution consistency for memory operations.

Inputs:

- ALU_Result: Memory address (computed in previous stage as base + offset).
- rs2_val: Store data (for S-type instructions, comes from register file earlier).
- MemRead, MemWrite: Control signals indicating load/store operation.
- rd: Destination register for loads.
- Instruction_Type: To differentiate ALU, load, store instructions.
- Instruction_Info: Full instruction bundle from previous stage.
- Flush: Cancel operation if a misprediction or exception occurred.

Outputs:

- Mem_Data_Out: Data read from memory (for loads).
- rd: Destination register for writeback.
- Mem_Access_Complete: Ready/valid signal for memory response.
- Instruction_Info: Passed forward to the Writeback stage with updated data.

Internal Components:

Data Memory

- Word-addressed memory array (supporting 32-bit accesses).
- Load (LW) and store (SW) instructions supported (aligned access assumed).

Access Control Logic:

- Reads data if MemRead is high.
- Writes data to address if MemWrite is high.

Logic:

For Load (LW):

- Access memory at ALU_Result
- Read 32-bit word → Mem_Data_Out
- Pass to Writeback stage to write to rd

For Store (SW):

- Access memory at ALU_Result
- Write rs2_val to address
- No writeback to register file needed

Hazards and control

Memory Dependency Hazards:

- In in-order pipelines, stores and loads from the same program are ordered, so hazard detection is minimal.

Alignment Faults:

- Ensure address is 4-byte aligned (lowest 2 bits of `ALU_Result` = 00).

Stalling:

- If memory is not ready (e.g., cache miss or blocking memory), stall pipeline.

Notes:

- Implement memory using reg [31:0] data_mem [0:MEM_SIZE-1]
- Use read/write control signals to gate memory access.
- Memory may need a separate clock domain or 1-cycle latency simulation.
- Include Mem_Access_Complete signal if you're supporting memory latency.

7. Writeback

Purpose:

- Commit the result of ALU or Load instructions to the **register file**.
- Mark the destination register as **ready** in the scoreboard (if used).
- Final step in the instruction lifecycle for in-order execution.

Inputs:

- rd: Destination register address.
- Write_Data: Result from ALU or Data Memory.
- RegWrite: Control signal (high for instructions that write to register file).
- MemToReg: Signal to select between ALU result and memory data.
- Instruction_Info: Full instruction context for bookkeeping and debug.
- Flush: Cancel writeback if the instruction was invalidated.

Outputs:

- RegFile[rd]: Register file gets updated with Write_Data.
- Scoreboard_Update: Destination register marked available.
- WB_Valid: Indicates successful commit.

Internal Components:

Register File (shared with decode stage):

- 1 or more write ports (2-way issue needs up to 2 writes/cycle).

Writeback Mux:

- Selects ALU result or Load data based on MemToReg.

Operation Logic:

If RegWrite = 1 and instruction is not flushed:

- Select write data:
 - ALU result for arithmetic/logical instructions
 - Memory result for loads
- Write Write_Data to register rd
- Mark rd as ready in the scoreboard

For store and branch instructions:

- RegWrite = 0, so no register is updated.

Consider:

- Writes must occur in program order to avoid WAW hazards.
- No register writes to x0 (hard-wired to zero) — must block these.
- In a 2-way design:
 - If both Instr1 and Instr2 write to the same register (rd1 = rd2), Instr2 must be stalled or dropped to maintain in-order correctness.

Notes:

- Register file writes occur on **positive clock edge** if RegWrite is asserted.
- Mux Write_Data = (MemToReg) ? Mem_Data_Out : ALU_Result.
- Scoreboard bit for rd is cleared (set to 0) to mark the register as available.

